

消耗品管理システム 本番環境移行手順

前提条件

- 本番PCにDockerがインストールされ、起動していること
- 本番PCにデータベース（PostgreSQL等）が設定済みであること
- 本番PCでポート8504が使用可能であること
- アプリケーションのソースコードが準備されていること

1. アプリケーションファイルの転送

1-1. 必要なファイルを本番PCに転送

以下のファイル・ディレクトリを本番PCの適切なディレクトリ（例: `/app/syomohin`）に転送します。

```
# 転送が必要なファイル・ディレクトリ
- app.py
- config.py
- database_manager.py
- email_sender.py
- pdf_generator.py
- permission_helper.py
- requirements.txt
- Dockerfile
- .dockerignore
- .env.example
- routes/
- templates/
- static/
- utils/
- scripts/
- resources/
- .streamlit/
```

1-2. 環境変数ファイルの作成

本番PC上で `.env` ファイルを作成します。

```
cd /app/syomohin
cp .env.example .env
```

`.env` ファイルを編集して、本番環境の設定を記述します。

```
# データベース接続情報（本番PCのデータベース設定に合わせる）
DB_HOST=localhost
```

```
DB_PORT=5432
DB_NAME=syomohin_db
DB_USER=syomohin_user
DB_PASSWORD=your_secure_password

# メール送信設定
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASSWORD=your-app-password
FROM_EMAIL=your-email@gmail.com
FROM_NAME=ダイソウ工業株式会社

# Flask設定
SECRET_KEY=your-production-secret-key-change-this
```

2. Dockerイメージのビルド

本番PC上でDockerイメージをビルドします。

```
cd /app/syomohin
docker build -t syomohin-app:latest .
```

ビルドには数分かかる場合があります。エラーが出た場合は、エラーメッセージを確認してください。

3. Dockerコンテナの起動

3-1. 初回起動（テスト）

まず、コンテナが正常に起動するかテストします。

```
docker run --rm -it \
  --name syomohin-app-test \
  -p 8504:8504 \
  --env-file .env \
  -v $(pwd)/uploads:/app/uploads \
  -v $(pwd)/data:/app/data \
  syomohin-app:latest
```

ブラウザで <http://本番PCのIPアドレス:8504> にアクセスして動作確認します。問題がなければ **Ctrl+C** で停止します。

3-2. 本番環境での起動（バックグラウンド実行）

本番環境では、コンテナをバックグラウンドで実行します。

```
docker run -d \  
  --name syomohin-app \  
  --restart unless-stopped \  
  -p 8504:8504 \  
  --env-file .env \  
  -v $(pwd)/uploads:/app/uploads \  
  -v $(pwd)/data:/app/data \  
  syomohin-app:latest
```

オプション説明:

- `-d`: バックグラウンドで実行
- `--name syomohin-app`: コンテナ名を指定
- `--restart unless-stopped`: サーバー再起動時に自動起動
- `-p 8504:8504`: ポート8504をホストに公開
- `--env-file .env`: 環境変数ファイルを読み込み
- `-v $(pwd)/uploads:/app/uploads`: アップロードファイルを永続化
- `-v $(pwd)/data:/app/data`: データファイルを永続化

3-3. Docker Compose を使用する場合（推奨）

`docker-compose.yml` を作成すると、より簡単に管理できます。

`docker-compose.yml`

```
version: '3.8'  
  
services:  
  app:  
    build: .  
    container_name: syomohin-app  
    restart: unless-stopped  
    ports:  
      - "8504:8504"  
    env_file:  
      - .env  
    volumes:  
      - ./uploads:/app/uploads  
      - ./data:/app/data  
    networks:  
      - syomohin-network  
  
networks:  
  syomohin-network:  
    driver: bridge
```

起動コマンド:

```
# ビルドと起動
docker-compose up -d --build

# ログの確認
docker-compose logs -f

# 停止
docker-compose down

# 再起動
docker-compose restart
```

4. 動作確認

4-1. コンテナの状態確認

```
# コンテナが起動しているか確認
docker ps

# ログを確認
docker logs syomohin-app

# リアルタイムでログを監視
docker logs -f syomohin-app
```

4-2. アプリケーションの動作確認

ブラウザで以下のURLにアクセスします。

```
http://本番PCのIPアドレス:8504
```

- ログイン画面が表示されるか確認
- ログイン後、各機能が正常に動作するか確認
- データベース接続が正常か確認

5. コンテナの管理コマンド

停止

```
docker stop syomohin-app
```

起動

```
docker start syomohin-app
```

再起動

```
docker restart syomohin-app
```

削除（停止後）

```
docker rm syomohin-app
```

イメージの更新（アプリケーション更新時）

```
# 既存コンテナを停止・削除
docker stop syomohin-app
docker rm syomohin-app

# イメージを再ビルド
docker build -t syomohin-app:latest .

# 新しいコンテナを起動
docker run -d \
  --name syomohin-app \
  --restart unless-stopped \
  -p 8504:8504 \
  --env-file .env \
  -v $(pwd)/uploads:/app/uploads \
  -v $(pwd)/data:/app/data \
  syomohin-app:latest
```

6. トラブルシューティング

コンテナが起動しない場合

```
# ログを確認
docker logs syomohin-app

# コンテナ内でシェルを起動して調査
docker exec -it syomohin-app /bin/bash
```

ポート8504が使用できない場合

```
# ポートが使用されているか確認 (Linux)
sudo netstat -tulpn | grep 8504

# または
sudo lsof -i :8504

# 使用中のプロセスを停止するか、別のポートを使用
docker run -d \
  --name syomohin-app \
  -p 8505:8504 \ # ホスト側を8505に変更
...
```

データベース接続エラーの場合

- `.env` ファイルのデータベース設定を確認
- 本番PCのデータベースが起動しているか確認
- データベースのホスト名・ポート番号が正しいか確認
- Dockerコンテナからデータベースにアクセスできるか確認

```
# コンテナ内からデータベースへの接続テスト
docker exec -it syomohin-app /bin/bash
apt-get update && apt-get install -y postgresql-client
psql -h DB_HOST -p DB_PORT -U DB_USER -d DB_NAME
```

ファイルのアップロードができない場合

- `uploads` ディレクトリのパーミッションを確認
- Docker ボリュームマウントが正しく設定されているか確認

```
# パーMISSIONの設定
chmod -R 755 uploads/
```

7. バックアップ

定期的にバックアップを取ることを推奨します。

```
# データベースのバックアップ (PostgreSQLの場合)
pg_dump -h localhost -U syomohin_user syomohin_db > backup_$(date +%Y%m%d).sql

# アップロードファイルのバックアップ
tar -czf uploads_backup_$(date +%Y%m%d).tar.gz uploads/
```

8. セキュリティ設定 (推奨)

HTTPS対応

本番環境では、HTTPSを使用することを強く推奨します。

1. SSL証明書を取得（Let's Encrypt等）
2. Nginxなどのリバースプロキシを設定
3. Docker Composeで Nginx コンテナを追加

ファイアウォール設定

```
# ポート8504への外部アクセスを許可（必要に応じて）
sudo ufw allow 8504/tcp
```

9. 監視とメンテナンス

ログローテーション

Dockerログが肥大化しないよう設定します。

```
# docker-compose.ymlに追加
services:
  app:
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

ヘルスチェック

定期的にアプリケーションの稼働状態を確認します。

```
# ヘルスチェックスクリプト例
curl -f http://localhost:8504/login || echo "アプリケーションが応答しません"
```

以上で、本番環境への移行が完了です。問題が発生した場合は、ログを確認して適切に対処してください。